

Escenario propuesto para el corte

Luego de un proceso arduo de selección y pruebas de conocimiento, usted es contratado como ingeniero junior por una de las instituciones más reconocidas a nivel nacional para el área de ingeniería biomédica. El objetivo del cargo es ejecutar actividades propias de su labor y que estén conforme a los procedimientos establecidos en el área de ingeniería, cumpliendo con los estándares de calidad planteados por la corporación.

Es de esta manera, su principal y primordial actividad es verificar y actualizar las hojas de vida de los diferentes equipos biomédicos con los que cuenta la institución, permitiendo verificar e identificar los documentos o soportes de gestión que hacen falta.

Para ello, usted como líder de equipo debe realizar e implementar un sistema de información (CRUD) que sea ágil y permita ser integrado a las herramientas office que maneja la institución. Esta característica es la más importante dado que toda la información se encuentra en archivos Excel.

Primera semana

Paso inicial

Es importante poder generar una ruta de trabajo la cual debe estar plasmada en un diagrama de actividades; por lo general se refleja cada una de estas actividades ejecutada por fases. En ese orden de ideas, es clave colocar en acción la metodología scrum para el desarrollo del problema propuesto.

¿Por qué es necesario tener una lógica de programación?

Conceptos básicos

Programa PSeint

PSeInt es una herramienta que sirve como apoyo a los estudiantes en sus primeros pasos en programación. Esta herramienta permite de manera simple e intuitiva desarrollar pseudolenguaje en español (complementado el proceso con un editor de diagramas de flujo), que sin duda alguna le permite entender los conceptos fundamentales de la algoritmia computacional, reduciendo las dificultades propias de un lenguaje y proporcionando un entorno de trabajo con algunas ayudas y recursos didácticos visuales.

Proceso de instalación

Ingrese a al siguiente link: <https://pseint.sourceforge.net/?page=descargas.php>



...una invitación a entrar en el maravilloso mundo de la programación...

[Portada](#) [Noticias](#) [Descargar](#) [Documentación](#) [Foros](#)

Última versión: 20210609



[Descargar Paquete para GNU/Linux 64bits](#) (tgz - 9.5MB)

[Descargar Paquete para GNU/Linux 32bits](#) (tgz - 9.6MB)



[Descargar Instalador para Microsoft Windows](#) (exe - 9.0MB)

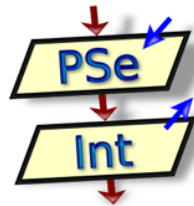
[Descargar Portable para Microsoft Windows](#) (zip - 12MB)



[Descargar Paquete para macOS \(64 bits\)](#) (tgz - 8.2MB)

(Ver versiones anteriores de 32bits)

Instalar - PSeInt



Bienvenido al asistente de instalación de PSeInt

Este programa instalará PSeInt versión 20210609 en su computadora.

Haga clic en Siguiente para continuar o en Cancelar para salir de la instalación.

Siguiente >

Cancelar

Instalar - PSeInt

Acuerdo de Licencia

Es importante que lea la siguiente información antes de continuar.



Por favor, lea el siguiente acuerdo de licencia. Debe aceptar las cláusulas de este acuerdo antes de continuar con la instalación.

PSeInt se distribuye bajo GNU General Public License.

GNU GENERAL PUBLIC LICENSE

Version 2, June 1991

Copyright (C) 1989, 1991 Free Software Foundation, Inc.
675 Mass Ave, Cambridge, MA 02139, USA

Everyone is permitted to copy and distribute verbatim copies
of this license document, but changing it is not allowed.

Preamble

The licenses for most software are designed to take away your
freedom to share and change it. By contrast, the GNU General Public
License is intended to guarantee your freedom to share and change free
software—to make sure the software is free for all its users. This

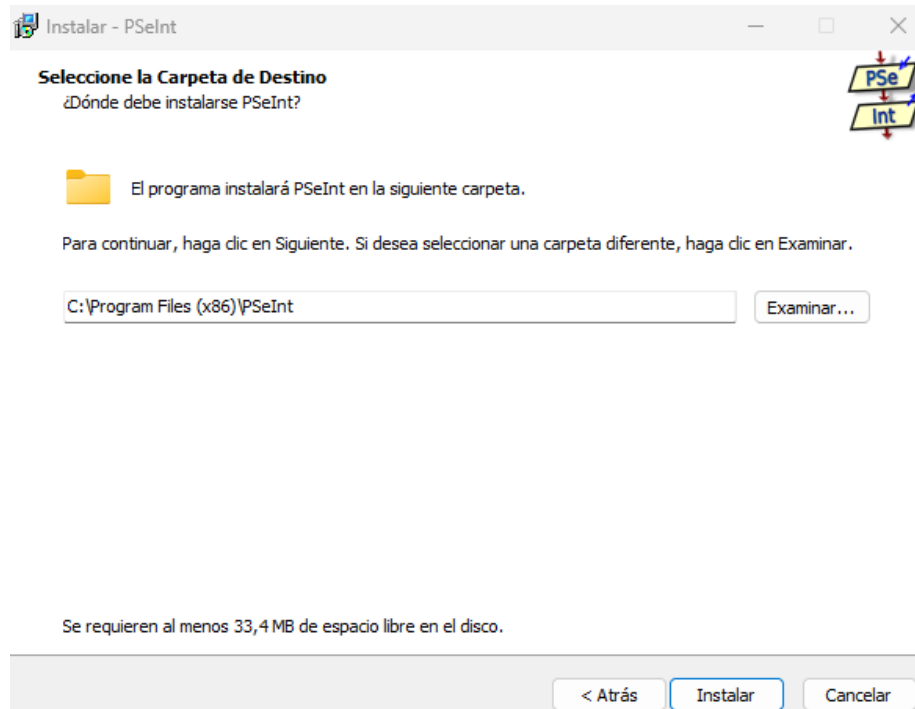
☒ Acepto el acuerdo

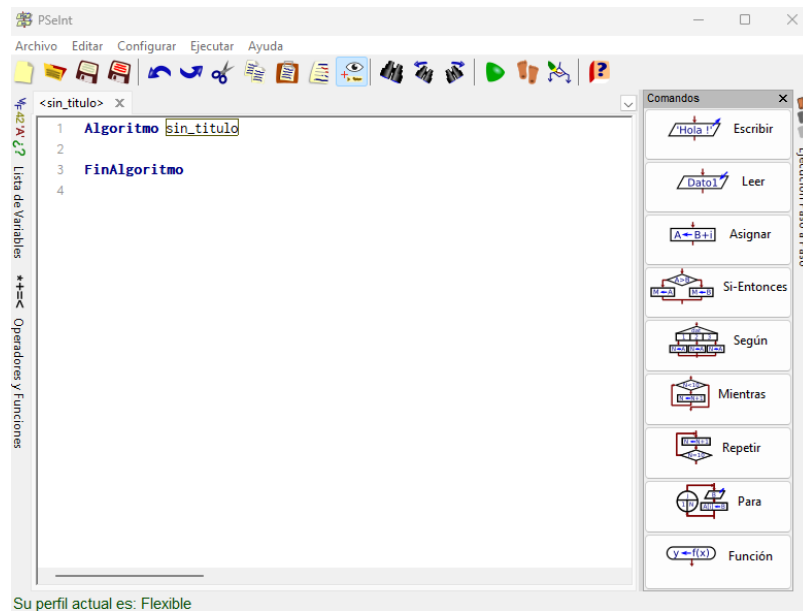
☐ No acepto el acuerdo

< Atrás

Siguiente >

Cancelar





Datos y variables

Los datos son las unidades de información con las que trabaja tu computadora para llevar a cabo una tarea en particular; es importante recordar que toda esta información se almacena de manera temporal en la memoria RAM.

Por lo general estos datos se almacenan en objetos llamados variables, constantes, vectores, celdas, entre otros. En ese sentido, es importante recordar que una variable constante siempre va a almacenar un dato de manera estática, es decir, no cambia su valor. Por ejemplo, el valor de pi o el valor de la raíz de 4.

Por otra parte, se comprende como una variable un valor no estático y que puede variar en cualquier momento. Por ejemplo, imaginemos una suma acumulada donde inicialmente una variable puede tener un valor de cero y a medida que valla cambiando en diferentes líneas de código podría ir cambiando su valor.

Existen algunas consideraciones a la hora de definir el nombre o identificador de una variable. En principio se debe iniciar con una letra, en lo posible en mayúscula y no es recomendable utilizar caracteres especiales, a excepción de guion bajo. De igual manera no se aceptan espacios entre los identificadores. En términos de extensión, se recomienda que no exceda de los 20 a 30 caracteres.

Finalmente, se recomienda que el nombre del identificador sea significativo, es decir, que tenga relación con el valor que está representado.

Algunos ejemplos:

Nombre Tiempo

Nombre_Tiempo

NombreTiempo

Tipos de datos

Carácter: almacenar cualquier texto o valor alfanumérico

Booleano: este tipo de variable solo podrá tener dos valores diferentes, verdadero o falso

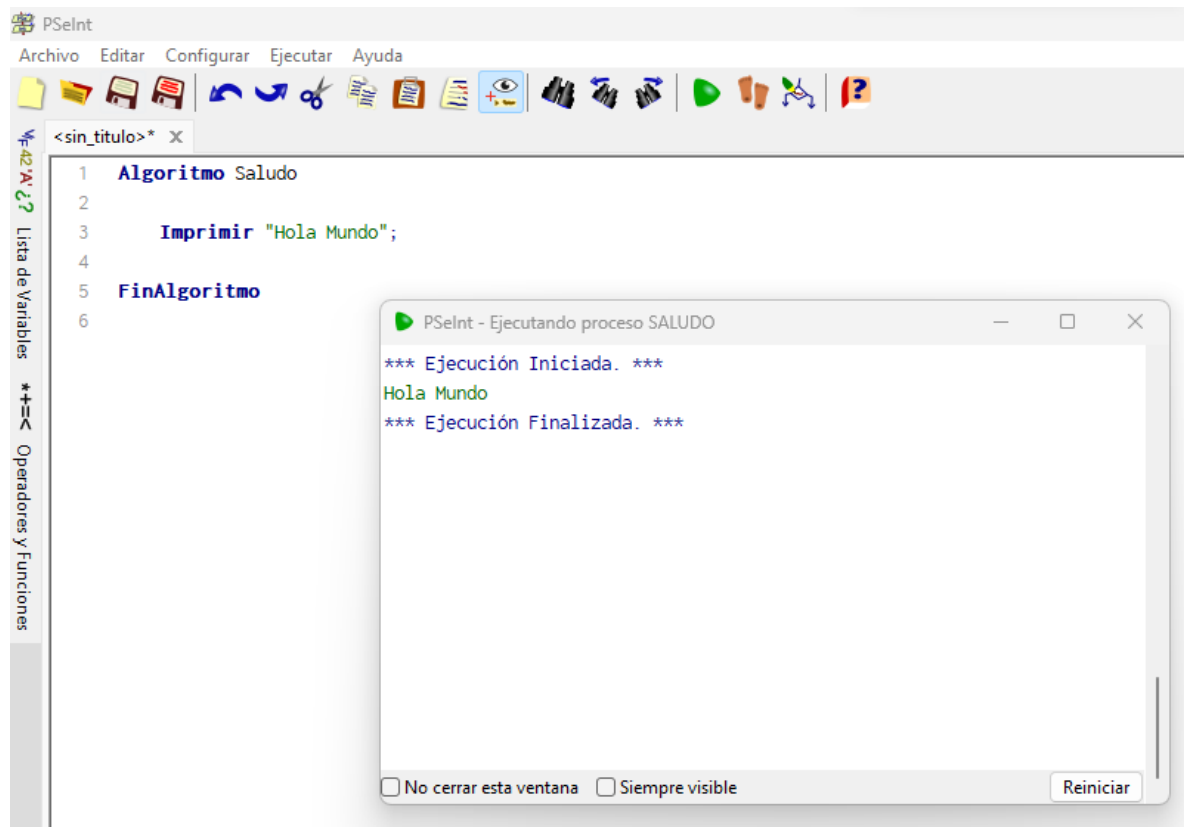
Real: son valores que tienen una parte fraccionaria o dicho de otra manera números decimales

Enteros: son datos que permiten representar números sin decimales; pueden ser positivos, negativos o en cero.

Ejercicio

Todo proceso tiene un inicio y un fin y en el programa PSeint no es diferente.

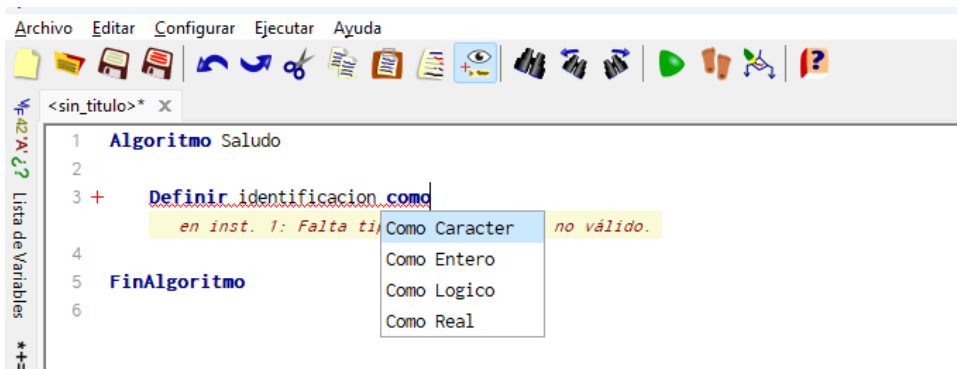
En ese sentido, vamos a imprimir nuestro primer “Hola Mundo”!.. sigue las siguientes instrucciones:



Asignación y asignación de variables

En el procedimiento de un algoritmo es clave ante todo definir y declarar las variables de trabajo, es decir, que no se puede asignar un valor a “X” a una variable si aún no ha sido declarada.

A continuación, en el programa PSeint se va a definir una variable a partir de la palabra reservada “definir identificación como” en donde:



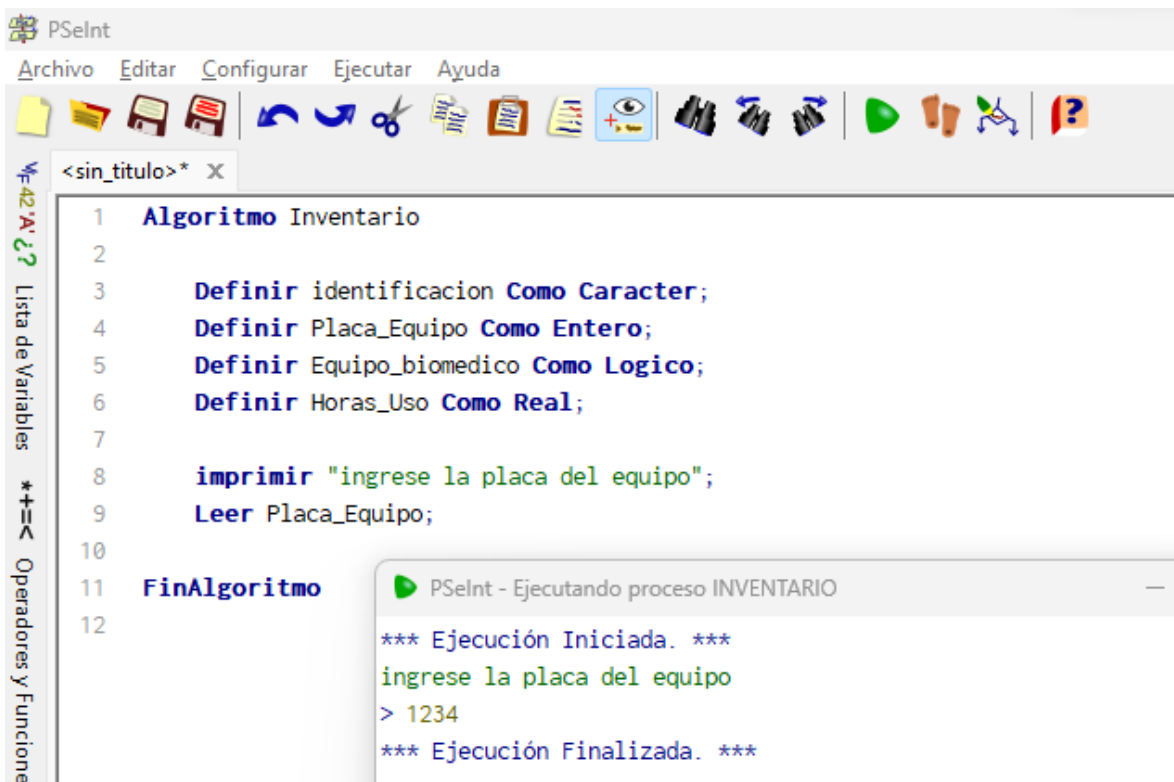
El tipo Carácter se utiliza cuando se desea almacenar en una variable un valor alfanumérico.

El tipo entero lo que hace es almacenar valores de tipo entero, es decir, uno, cinco, diez, sin tener en cuenta la parte decimal.

El tipo lógico almacenaría valores propiamente de verdadero y falso.

El tipo real almacena parte de los números enteros y decimales.

A continuación, un ejemplo:



Operadores

Existen tres tipos de operadores principales los cuales son Aritméticos, Relacionales y Lógicos.

Los operadores aritméticos son un conjunto de operadores que se utilizan para realizar cálculos matemáticos como suma, resta, multiplicación y división. Sin embargo, en estas operaciones existen ciertas prioridades como se observa en la siguiente tabla:

Operador	Uso	Prioridad
^	Exponente	1
*	Multiplicación	2
/	División	2
Mod	Residuo	2
+	Suma	3
-	Resta	3

Los operadores relacionales cumplen una función especial de evaluar si una expresión es válida o no. Sin embargo, en estas operaciones existen algunas características como se observa en la siguiente tabla:

Operador	Uso	Ejemplo
<	Menor que	A	Mayor que	A>B
=	Igual a	A=B
<>	Diferente a	A<>B
<=	Menor o igual que	A<=B
>=	Mayor o igual que	A>=B

Finalmente, se tiene los operadores lógicos los cuales son utilizados en informática para evaluar el cumplimiento de dos o más condiciones. De esa manera se puede decir que existen tres operadores principalmente: And, Or y Not.

And		
Condición 1	Condición 2	Condición 3
Verdadero	Verdadero	Verdadero
Verdadero	Falso	Falso
Falso	Verdadero	Falso
Falso	Falso	Falso

Or		
Condición 1	Condición 2	Condición 3
Verdadero	Verdadero	Verdadero
Verdadero	Falso	Verdadero
Falso	Verdadero	Verdadero
Falso	Falso	Falso

Not	
Condición 1	Resultado
Verdadero	Falso
Falso	Verdadero

Entrada y salida de datos

Para comprender mejor de que se trata el manejo de datos, se va a realizar un sencillo ejercicio en PSeint, con lo cual se pretende familiarizar el uso de variables y con ello los datos que se van a tratar.

The screenshot shows the PSeint IDE with a code editor and an execution window. The code defines two real variables, NumeroUno and NumeroDos, and performs basic arithmetic operations: addition, multiplication, subtraction, and division. The execution window shows the program running with input values 7 and 3, displaying the results of each operation.

```

1  Algoritmo Operaciones_basicas
2
3  Definir NumeroUno Como Real;
4  Definir NumeroDos Como Real;
5
6  Imprimir "Digite el primer numero";
7  Leer NumeroUno;
8
9  Imprimir "Digite el segundo numero";
10 Leer NumeroDos;
11
12 Imprimir "La suma de ", NumeroUno, " mas el ", NumeroDos, " da como resultado ", NumeroUno + NumeroDos;
13 Imprimir "La multiplicacion de ", NumeroUno, " mas el ", NumeroDos, " da como resultado ", NumeroUno * NumeroDos;
14 Imprimir "La resta de ", NumeroUno, " mas el ", NumeroDos, " da como resultado ", NumeroUno - NumeroDos;
15 Imprimir "La division de ", NumeroUno, " mas el ", NumeroDos, " da como resultado ", NumeroUno / NumeroDos;
16
17 FinAlgoritmo
18

```

Execution Output:

```

*** Ejecución Iniciada. ***
Digite el primer numero
> 7
Digite el segundo numero
> 3
La suma de 7 mas el 3 da como resultado 10
La multiplicacion de 7 mas el 3 da como resultado 21
La resta de 7 mas el 3 da como resultado 4
La division de 7 mas el 3 da como resultado 2.333333333
*** Ejecución Finalizada. ***

```

Es importante tener en cuenta que solamente se puede realizar un proceso por cada una de las ventanas de PSeint.

Por tal razón es necesario abrir otra ventana para realizar cualquier otro ejercicio.

Ahora veamos cómo se puede implementar un sencillo algoritmo para determinar el promedio de las medidas obtenidas en un proceso de calibración de un sensor de temperatura.

The screenshot displays the PSeInt IDE interface. The main editor window contains the following code:

```
1 // proceso calibración de un sensor
2 Algoritmo Muestras_Temperatura
3
4     Definir Valor1, Valor2, Valor3, Valor4 Como Real;
5
6     Imprimir Sin Saltar "Registre el primer valor";
7     Leer Valor1;
8     Imprimir Sin Saltar "Registre el segundo valor";
9     Leer Valor2;
10    Imprimir Sin Saltar "Registre el tercer valor";
11    Leer Valor3;
12    Imprimir Sin Saltar "Registre el cuarto valor";
13    Leer Valor4;
14
15    Imprimir "El valor promedio del proceso de calibración del sensor es: ", (Valor1 + Valor2 + Valor3 + Valor4) / 4;
16
17 FinAlgoritmo
18
```

On the left side, there is a vertical toolbar with icons for 'Lista de Variables' and 'Operadores y Funciones'. A small window titled 'PSeInt - Ejecutando proceso MUESTRAS_TEMPERATURA' is overlaid on the bottom right, showing the execution output:

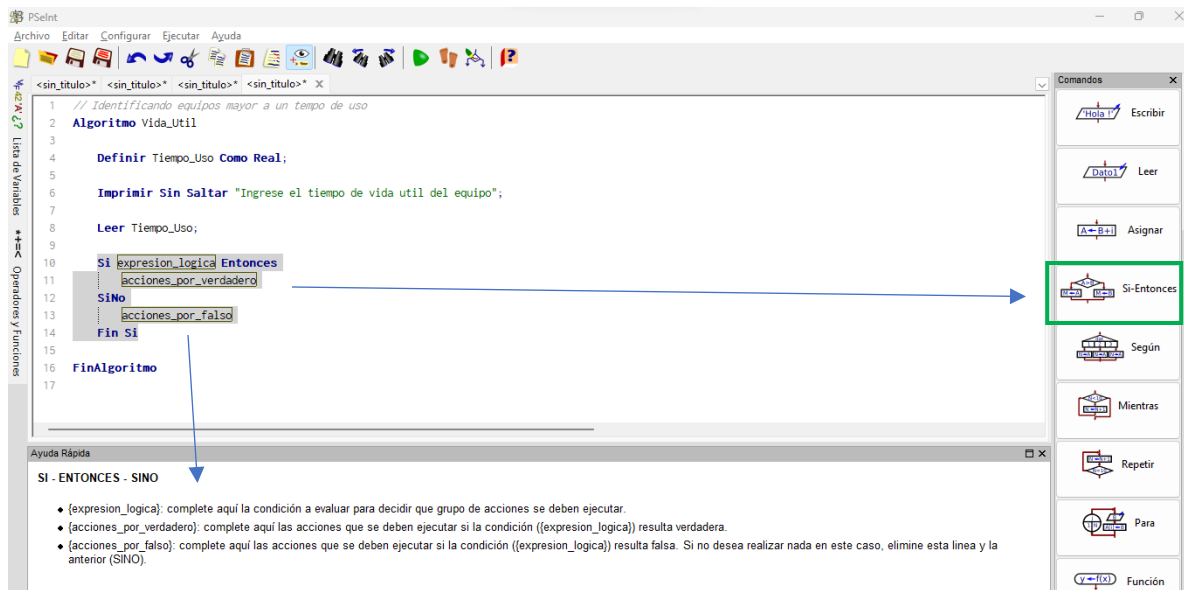
```
*** Ejecución Iniciada. ***
Registre el primer valor> 2.5
Registre el segundo valor> 3
Registre el tercer valor> 4.5
Registre el cuarto valor> 2
El valor promedio del proceso de calibración del sensor es: 3
*** Ejecución Finalizada. ***
```

Condiciones Simples

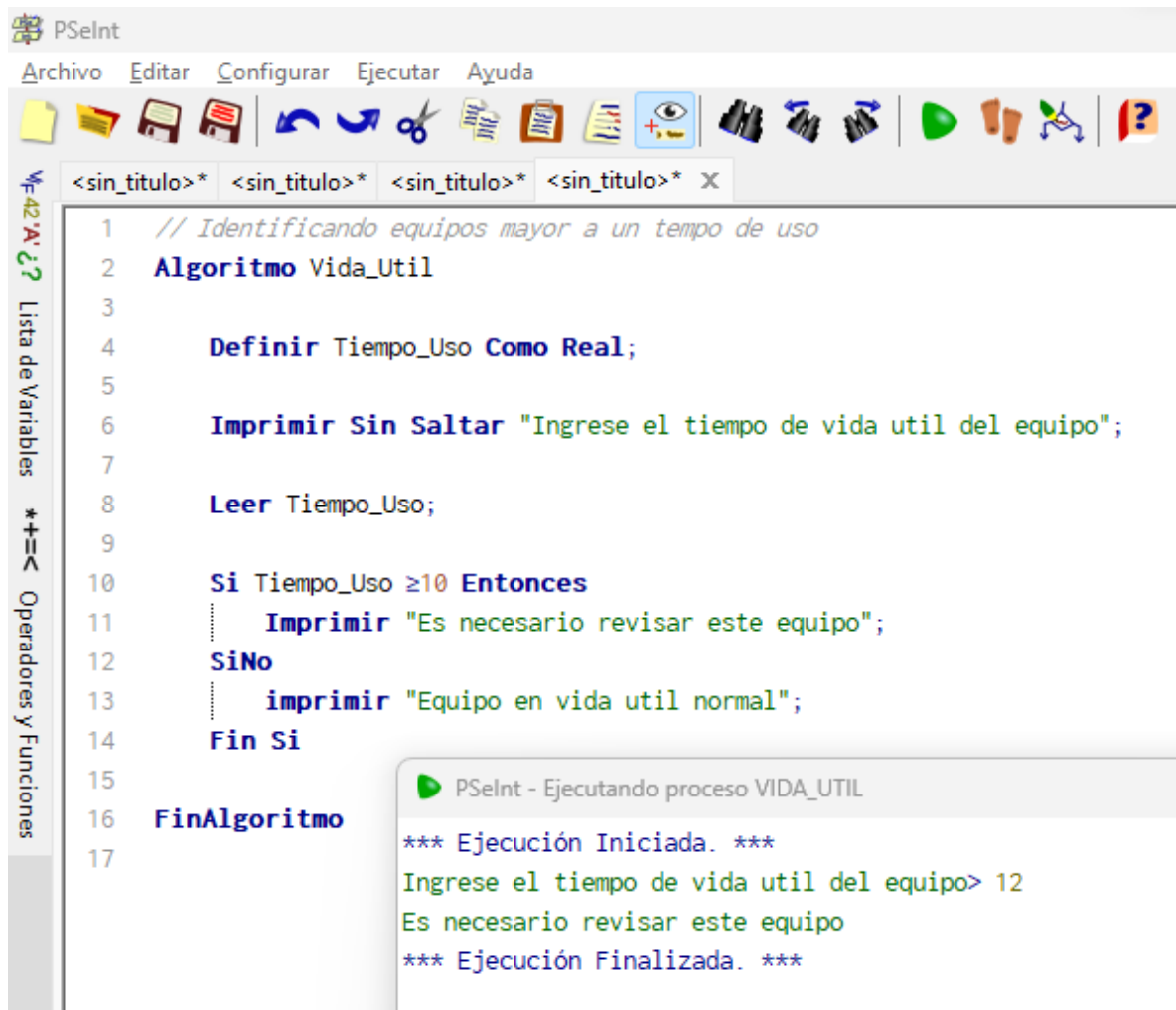
La importancia de comprender como utilizar de manera práctica las diferentes estructuras condicionales en programación es clave para solucionar diferentes problemas en el campo de la ingeniería.

Empezaremos por abrir una nueva hoja de trabajo y definir nuestras variables de trabajo; en esta ocasión la idea es utilizar una estructura condicional para identificar equipos mayores a cierto tiempo para una posible renovación.

En este caso se hará uso del condicional **si-entonces** para determinar cierta condición.

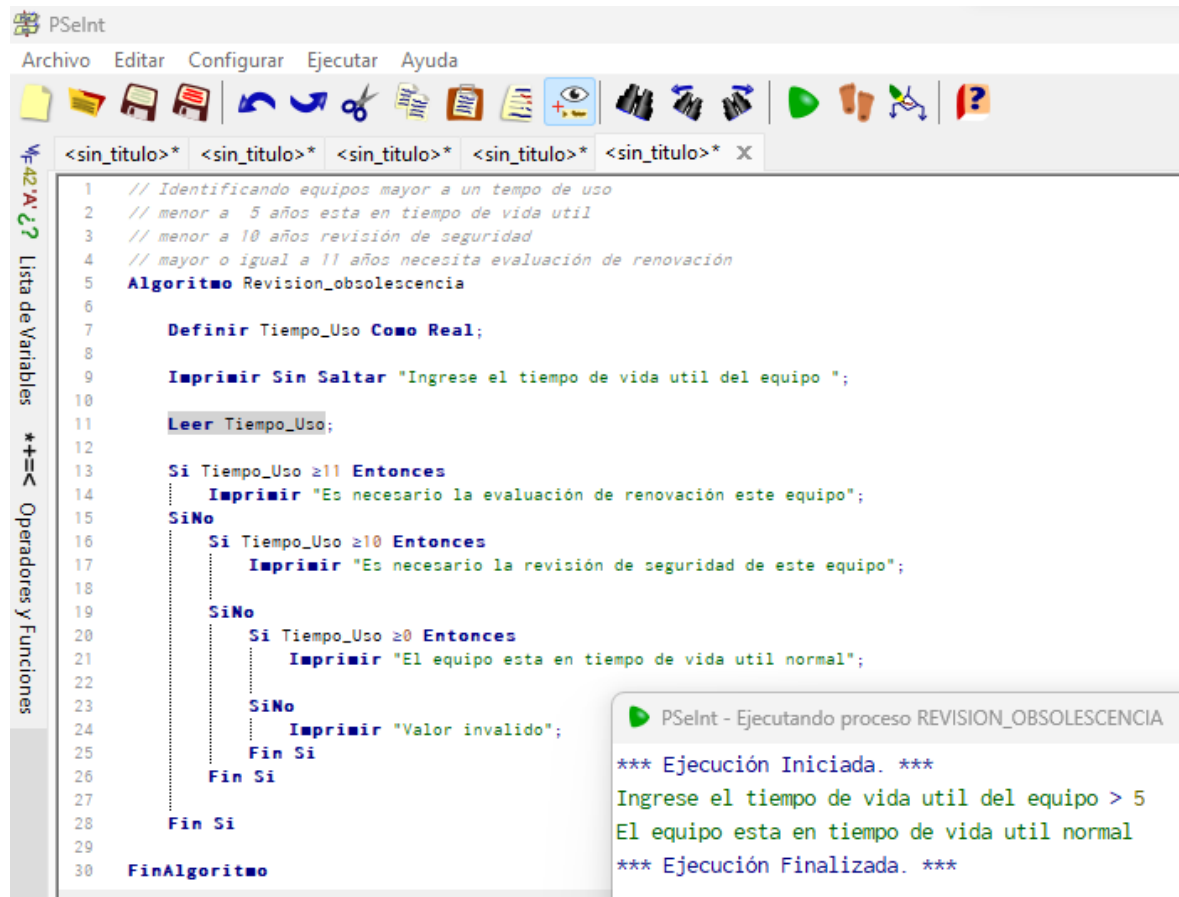


De esta manera se puede implementar de la siguiente forma:



Condiciones anidadas

Las condiciones anidadas se refieren al uso de una o más condiciones dentro de otra. En algunas ocasiones pueden ser remplazadas por condicionales compuestas o estructuras de selección múltiple. Recordemos que todas las condiciones tienen un inicio y un fin, de no ser así, puede generar un error en tiempos de ejecución.



```
1 // Identificando equipos mayor a un tiempo de uso
2 // menor a 5 años esta en tiempo de vida util
3 // menor a 10 años revisión de seguridad
4 // mayor o igual a 11 años necesita evaluación de renovación
5 Algoritmo Revision_obsolescencia
6
7     Definir Tiempo_Uso Como Real;
8
9     Imprimir Sin Saltar "Ingrese el tiempo de vida util del equipo ";
10
11     Leer Tiempo_Uso;
12
13     Si Tiempo_Uso ≥ 11 Entonces
14         Imprimir "Es necesario la evaluación de renovación este equipo";
15     SiNo
16         Si Tiempo_Uso ≥ 10 Entonces
17             Imprimir "Es necesario la revisión de seguridad de este equipo";
18         SiNo
19             Si Tiempo_Uso ≥ 0 Entonces
20                 Imprimir "El equipo esta en tiempo de vida util normal";
21             SiNo
22                 Imprimir "Valor invalido";
23             Fin Si
24         Fin Si
25     Fin Si
26
27     Fin Si
28
29     FinAlgoritmo
```

PSeInt - Ejecutando proceso REVISION_OBSOLESCENCIA

*** Ejecución Iniciada. ***

Ingrese el tiempo de vida util del equipo > 5

El equipo esta en tiempo de vida util normal

*** Ejecución Finalizada. ***

Otro ejercicio más... ahora validaremos un número en particular para saber si es positivo, negativo o es un valor de cero.

```
1 //evaluar un número si es positivo, negativo o tiene un valor de cero
2 Algoritmo DeterminarNumero
3
4     Definir Numero Como Real;
5     Definir ResultadoNumero Como Caracter;
6
7     Imprimir Sin Saltar "digite un numero ";
8     Leer Numero;
9
10    Si Numero>0 Entonces
11        ResultadoNumero="positivo";
12
13    SiNo
14        Si Numero=0 Entonces
15            ResultadoNumero="El valor es cero";
16
17        SiNo
18            ResultadoNumero="Negativo";
19        Fin Si
20
21    Fin Si
22    Imprimir ResultadoNumero;
23
24 FinAlgoritmo
```

PSeInt - Ejecutando proceso DETERMINARNUMERO

```
*** Ejecución Iniciada. ***
digite un numero > -7
Negativo
*** Ejecución Finalizada. ***
```

Condicionales compuestos

Este tipo de estructuras evalúan más de una condición en una sola expresión. Para ello se debe tener en cuenta otros elementos importantes, como los conocidos operadores lógicos. Es necesario recordar la tabla de verdad que condiciona el funcionamiento de estos.

A manera de ejemplo simulemos la comparación de tres números de placa de equipos con la finalidad de conocer si están repetidos o no.

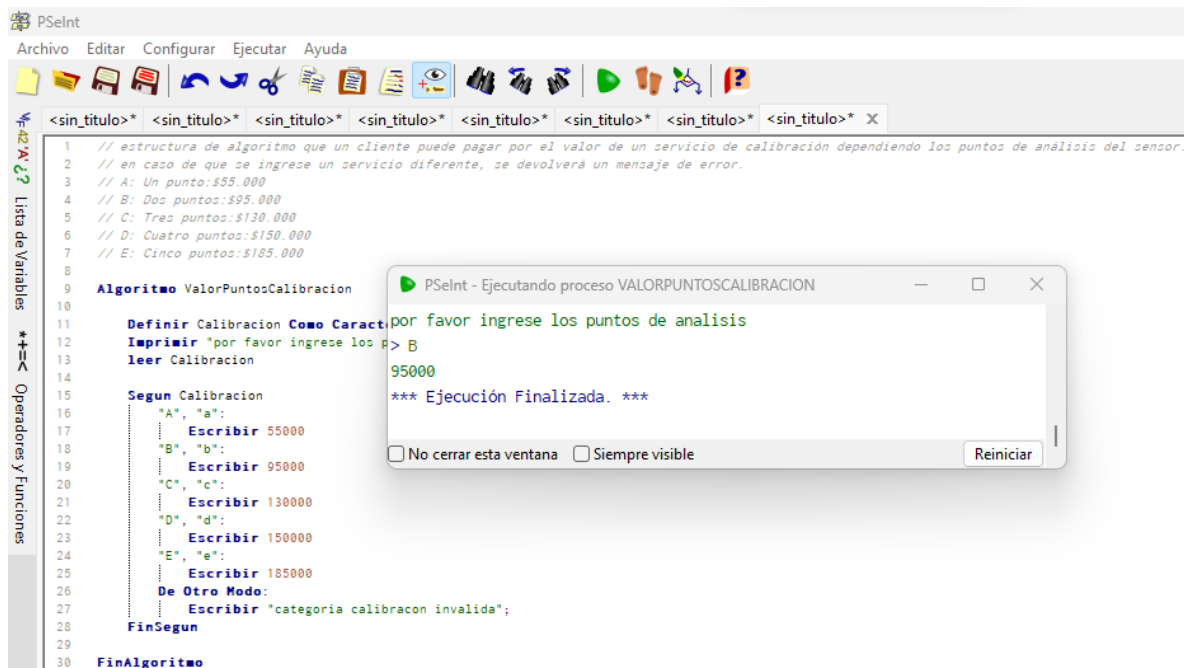
```
1 // Algoritmo que valida 3 números de placa y verifica si son iguales o diferentes
2
3 Algoritmo EvaluarNumerosPlacas
4
5     Definir Placa1, Placa2, Placa3 Como Real;
6     Imprimir "Ingrese los numeros a validar";
7     Leer Placa1, Placa2, Placa3;
8
9     Si Placa1=Placa2 y Placa1=Placa3 Entonces
10         Imprimir "Todos los numeros de placas son iguales!";
11     SiNo
12         Si Placa1=Placa2 o Placa1=Placa3 o Placa2=Placa3 Entonces
13             Imprimir "Existe dos numeros de placas iguales";
14         SiNo
15             Imprimir "Todos los numeros de placas son diferentes!";
16         Fin Si
17     Fin Si
18
19 FinAlgoritmo
20
```

*** Ejecución Iniciada. ***
Ingrese los numeros a validar
> 4
> 5
> 6
Todos los numeros de placas son diferentes!
*** Ejecución Finalizada. ***

Selección múltiple

Tiene la utilidad de validar varias selecciones y es muy parecido a un condicional anidado, siempre y cuando lo que se vaya a evaluar sea una sola variable. Esta estructura condicional por lo general se utiliza cuando se tiene o aplica muchas condiciones en una variable.

Un ejemplo de ello:



¿Ejercicio... como se podría consultar en que mes de año estaría programado una calibración para un equipo en particular?

Ciclos

Existen tres tipos de ciclos básicos, el “para”, el “mientras” y el “hacer mientras” o repetir algún ciclo. En ese sentido, el ciclo para se utiliza cuando se conoce la cantidad de iteraciones que se debe realizar y por lo general va en conjunto con una variable “contadora” la cual incrementa o disminuye un valor de iteraciones.

A continuación, se presenta un ejemplo sencillo sobre la ganancia que puede dejar un equipo biomédico a partir de su valor inicial de adquisición.

PSelnt

Archivo Editar Configurar Ejecutar Ayuda

<sin_titulo>* <sin_titulo>* <sin_titulo>* <sin_titulo>* <sin_titulo>* <sin_titulo>* <sin_titulo>* <sin_titulo>* <sin_titulo>* <sin_titulo>* X

```
1 // se desea conocer cuál sería el retorno de la inversión por la compra de un equipo biomédico al pasar el tiempo.
2 // para ello se discrimina en la cantidad de meses que se desea conocer las "ganancias" por la prestación del servicio.
3 Algoritmo DepreciaciónEquipo
4
5     Definir ValorCompra Como Real;
6     Definir ValorAcumulado Como Real;
7     Definir CantidadMeses Como Entero;
8
9     Imprimir "Digite el valor del Equipo";
10    Leer ValorCompra;
11
12    Imprimir "Ingrese la cantidad de meses";
13    leer CantidadMeses;
14
15    ValorAcumulado=ValorCompra;
16    Para X=1 Hasta CantidadMeses
17        ValorAcumulado=(ValorAcumulado* 0.03)+ValorAcumulado;
18        Imprimir "El valor para el mes ", X, " es de ", ValorAcumulado;
19    FinPara
20
21    Imprimir "La ganancia que deja el equipo comprado inicialmente por $", ValorCompra, " es de ", ValorAcumulado;
22
23 FinAlgoritmo
24
```

Lista de Variables

Operadores y Funciones

PSelnt - Ejecutando proceso DEPRECIACIÓN EQUIPO

*** Ejecución Iniciada. ***

Digite el valor del Equipo

> 45000000

Ingrese la cantidad de meses

> 12

El valor para el mes 1 es de 46350000

El valor para el mes 2 es de 47740500

El valor para el mes 3 es de 49172715

El valor para el mes 4 es de 50647896.450000003

El valor para el mes 5 es de 52167333.343500003

El valor para el mes 6 es de 53732353.343805

El valor para el mes 7 es de 55344323.944119148

El valor para el mes 8 es de 57004653.662442721

El valor para el mes 9 es de 58714793.272316001

El valor para el mes 10 es de 60476237.07048548

El valor para el mes 11 es de 62290524.182600044

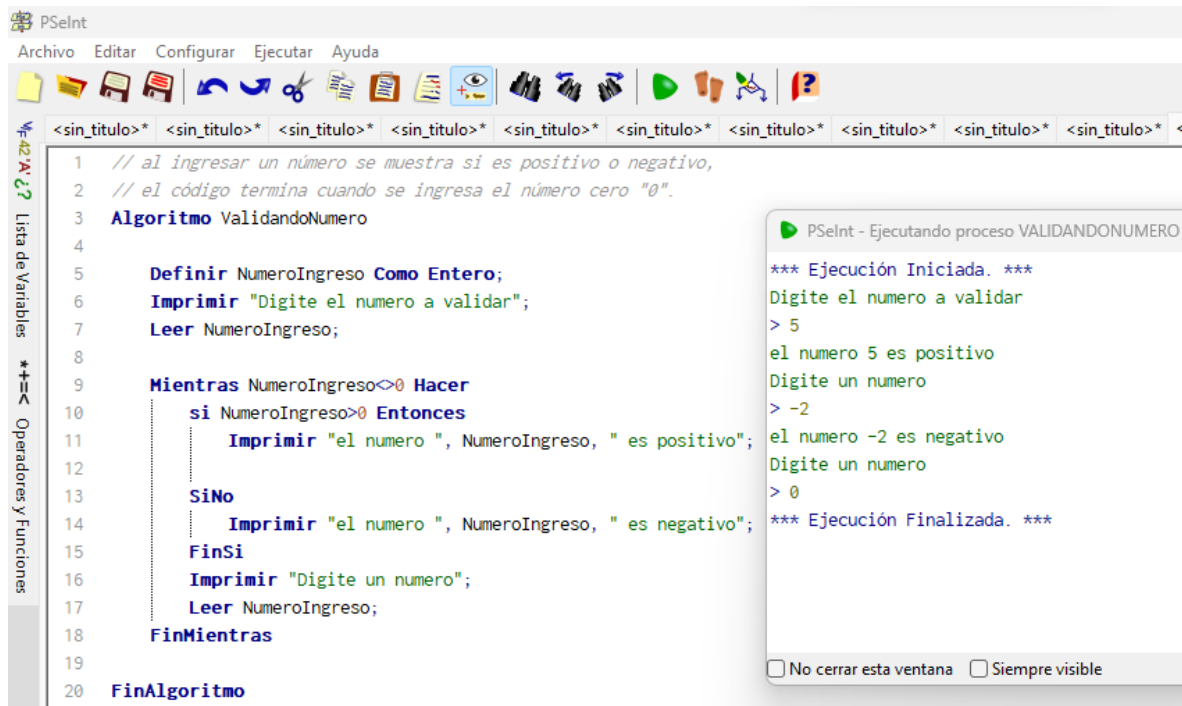
El valor para el mes 12 es de 64159239.908078045

La ganancia que deja el equipo comprado inicialmente por \$45000000 es de 64159239.908078045

Ciclo mientras

Este ciclo ejecuta un bloque de instrucciones, siempre y cuando una condición sea verdadera; en dado caso que la condición sea falsa, el ciclo se deja de ejecutar y continua con el recorrido de las líneas de código. Vale la pena resaltar que este ciclo se puede repetir muchas veces.

Un ejemplo de lo anterior es la validación de un número positivo, negativo o detener un proceso al ingresar el número cero (0).



```
1 // al ingresar un número se muestra si es positivo o negativo,
2 // el código termina cuando se ingresa el número cero "0".
3 Algoritmo ValidandoNumero
4
5     Definir NumeroIngreso Como Entero;
6     Imprimir "Digite el numero a validar";
7     Leer NumeroIngreso;
8
9     Mientras NumeroIngreso<>0 Hacer
10         si NumeroIngreso>0 Entonces
11             Imprimir "el numero ", NumeroIngreso, " es positivo";
12
13         SiNo
14             Imprimir "el numero ", NumeroIngreso, " es negativo";
15         FinSi
16         Imprimir "Digite un numero";
17         Leer NumeroIngreso;
18     FinMientras
19
20 FinAlgoritmo
```

PSeInt - Ejecutando proceso VALIDANDONUMERO

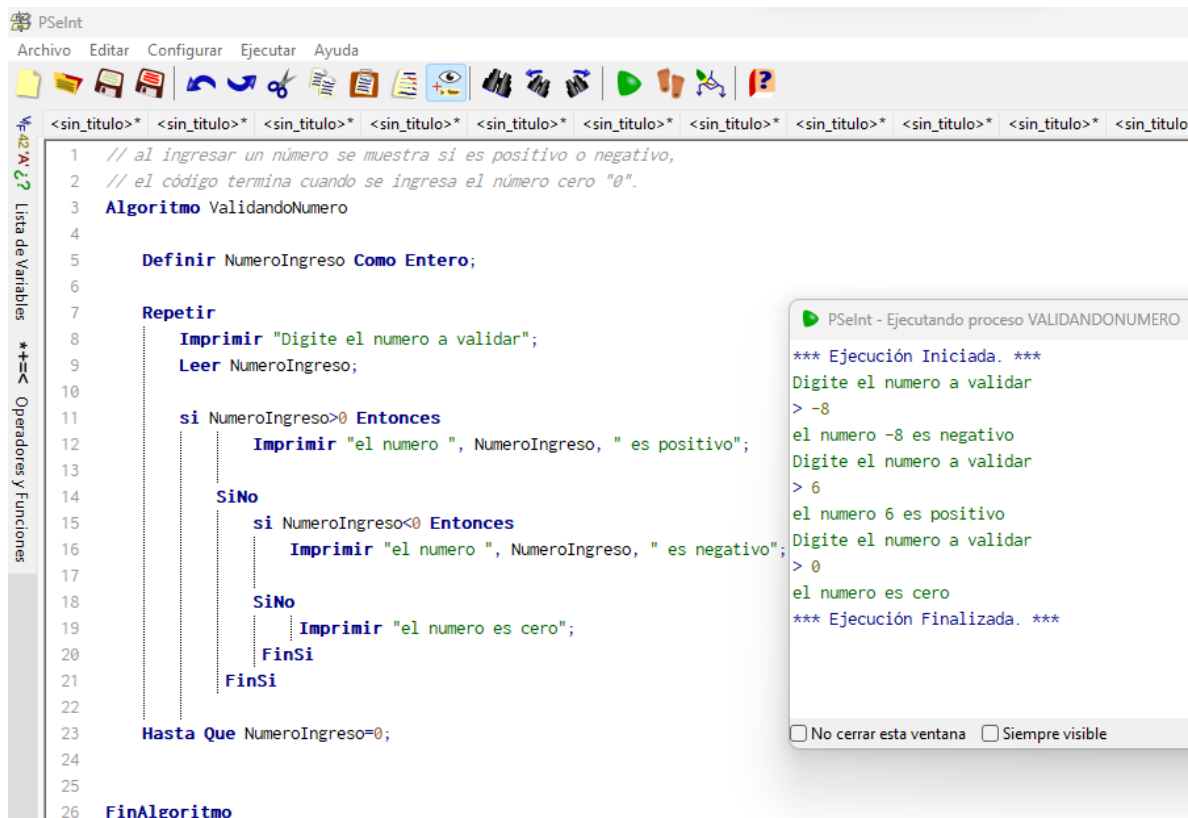
```
*** Ejecución Iniciada. ***
Digite el numero a validar
> 5
el numero 5 es positivo
Digite un numero
> -2
el numero -2 es negativo
Digite un numero
> 0
*** Ejecución Finalizada. ***
```

☐ No cerrar esta ventana ☐ Siempre visible

Ciclo repetir

El ciclo “repetir” es muy similar al ciclo “mientras” sin embargo, tiene una pequeña diferencia y es que la condición no se encuentra al principio del algoritmo sino al final de este y se ejecuta hasta que una variable tome un valor determinado en la condición inicial.

Un ejemplo de acuerdo con el anterior punto.



Hasta esta parte, la idea es que hayas aprendido o en el mejor de los casos recordado algunos conceptos importantes en términos de la programación básica.

¡Es hora de entrar en materia!

Programación Visual Basic

Con el uso de visual Basic se puede generar nuevos algoritmos para analizar y manipular datos, crear herramientas con el objetivo de optimizar procesos o implementar aplicaciones a la medida de las necesidades.

Es importante tener en cuenta que cada elemento de Excel es representado en Visual Basic como un objeto, por ejemplo, un workbook que representa un libro o un sheet que representa una hoja, un char a un gráfico. En ese mismo aspecto, cada uno de estos objetos tienen ciertas propiedades y métodos.

Un ejemplo de lo anterior es, considerar un automóvil como objeto y sus características como su marca, modelo y color. De igual forma, se puede decir que existen otras propiedades que explican o dan cuenta del estado del vehículo como: el estado del motor, kilometraje, nivel de gasolina entre otros.

De otro lado se tienen los métodos que representan las acciones que se pueden realizar con el automóvil como es el caso de encenderlo, marchar hacia adelante o hacia atrás o frenar.

Bajo esta lógica se puede relacionar el funcionamiento de Visual Basic con el automóvil, siendo una celda capaz de tener ciertas propiedades como el número de fila y columna donde se ubica la celda como tal. Ahora, el valor que contiene la celda, el color, tipo y tamaño de letra entre otras muchas propiedades.

En términos de métodos se puede comparar con las acciones que se pueden hacer con este objeto, es decir, cuando se selecciona, modifica o combina varias celdas.

¿Qué es una Macro en Excel?

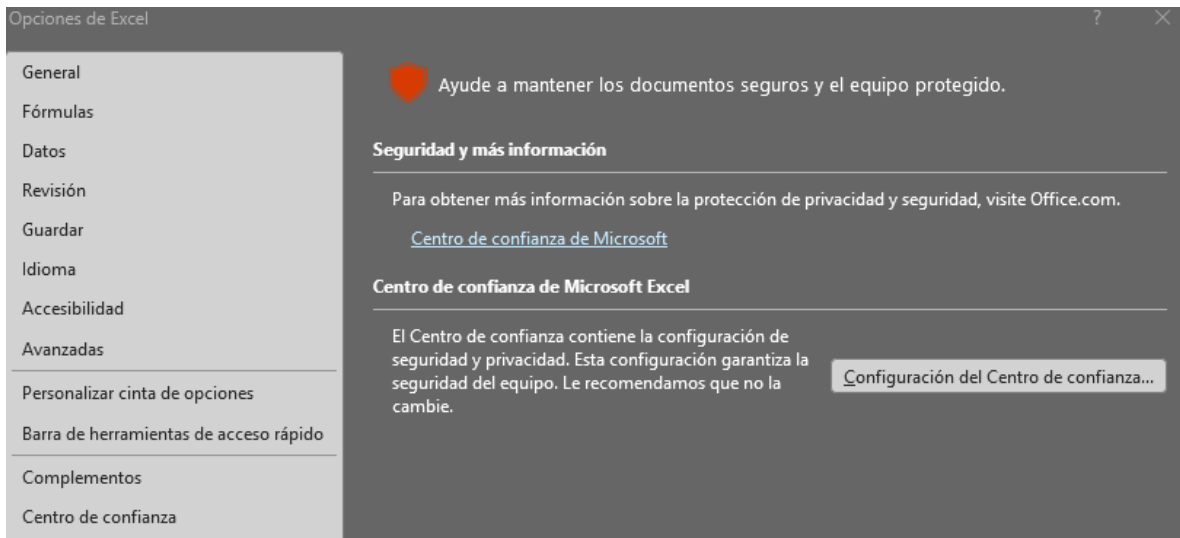
Se puede definir como un conjunto de instrucciones que se almacenan en un módulo dentro de la herramienta “programador”. Todas las macros de Excel se describen en un lenguaje particular nacido como Visual Basic para realizar aplicaciones, este lenguaje permite acceder y utilizar cada uno de los objetos con sus respectivas propiedades, métodos y eventos.

Establecer seguridad en macros en Excel

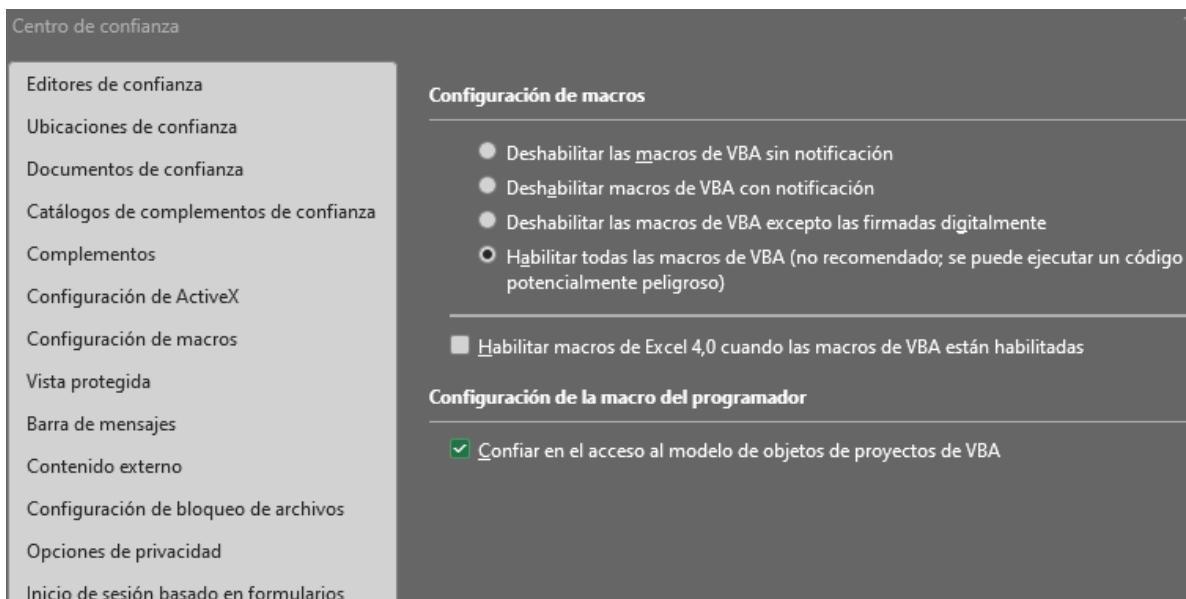
A la hora de abrir un libro que contenga una macro con código peligroso, podría causar algún tipo de daño en el pc entre otros problemas.

Para ello Excel tiene una configuración inicial donde se bloquean la ejecución de macros de manera automática. No obstante, a la hora de crear una macro y no deseas tener conflictos por tal protección, es necesario modificar la configuración de seguridad tal como se presenta en los siguientes pasos:

- Abrir un archivo de Excel nuevo
- En la parte superior del menú, haga clic en archivo
- En la parte inferior final haga clic en opciones, luego en centro de confianza y finalmente en configuración del centro de confianza.



- Busque y haga clic en configuración de macros y seleccione la última opción del menú que se muestra al lado derecho de la pantalla. Por otra parte, seleccione la opción de “confiar en el acceso al modelo de objetos de proyectos de VBA”.



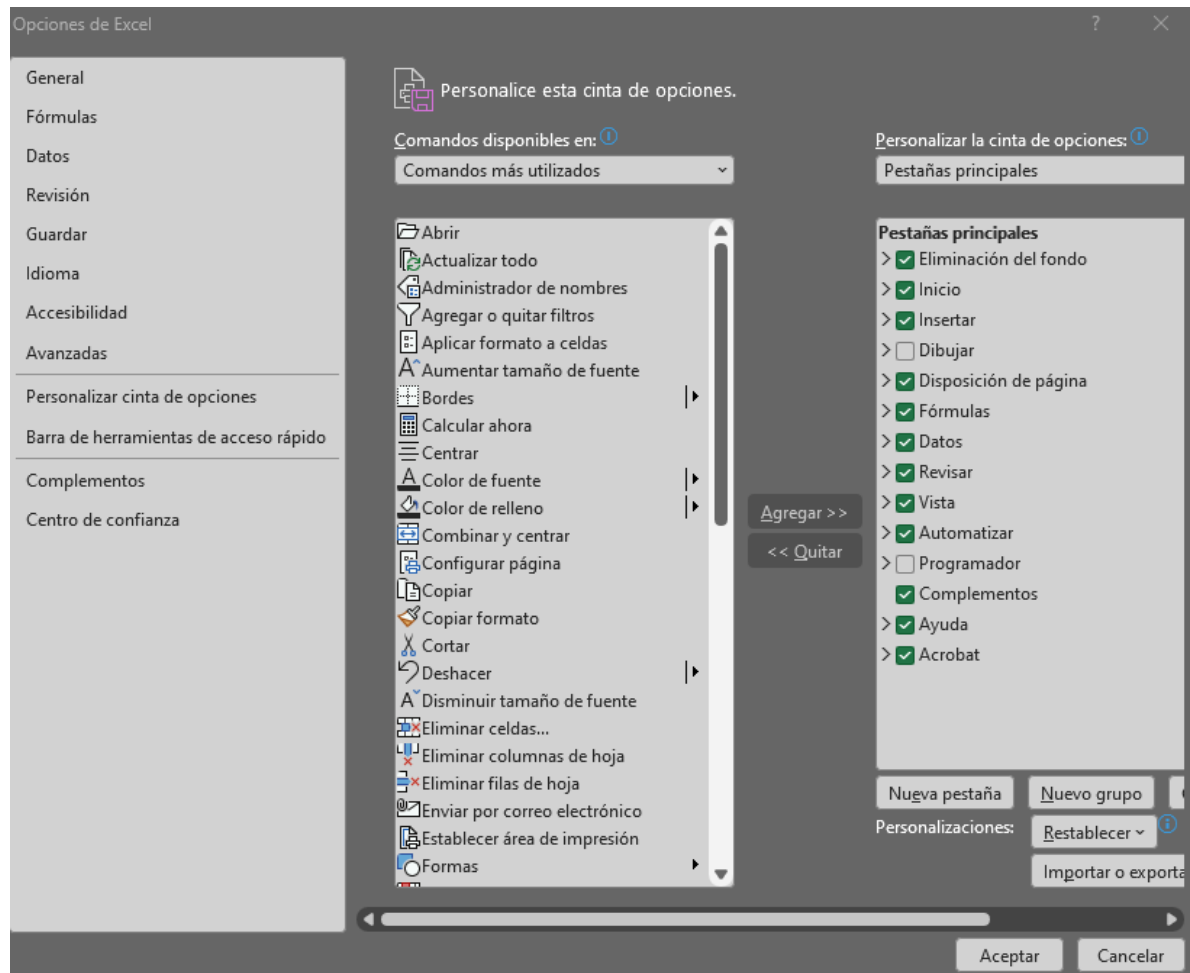
Finalmente, haga clic en aceptar hasta salir del menú.

Habilitar y mostrar la vista de programador

Es de suma importancia y necesario poder habilitar la pestaña en el menú del modo programador, en la cual se puede desarrollar diferentes aplicaciones. En ese sentido, siga los siguientes pasos:

Haga clic en alguna de las opciones del menú, como por ejemplo “vista” y haga clic derecho sobre ella. Paso seguido haga clic en “personalizar la cinta de opciones”.

En la parte derecha busque y seleccione la opción “Programador y/o Desarrollador”. Acto seguido haga clic en aceptar.



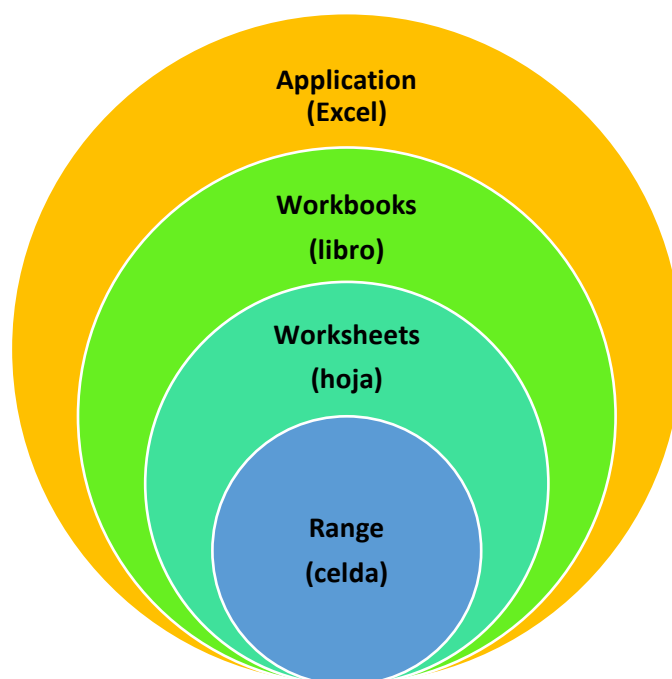
Editor de Visual Basic

Es un programa independiente de Excel pero que está estrechamente relacionado a él, ya que permite escribir código para el desarrollo de aplicaciones o lo que se conoce como macros.

Para ello vamos al menú principal y seleccionamos la opción de programador. Una vez en el modo programador, haga clic en la primera opción “Visual Basic” donde se abrirá el editor para comenzar a crear los diferentes programas, aplicaciones o formularios.

Cada grupo debe enviar un video no mayor a 5 minutos explicando de forma general el entorno de VBA (cada opción del menú / video)

Una macro es definida como un conjunto de instrucciones que se encuentran almacenadas y listas para ser ejecutadas en cualquier momento a través de un comando o el teclado. Estas instrucciones deben ser escritas en un lenguaje de programación que sea comprendido por el editor de VBA. En este orden de ideas, VBA es un entorno de programación que fue pensado para que los desarrolladores puedan crear soluciones.



Una de las claves para dominar las macros en Excel está principalmente asociado con el buen uso del lenguaje de programación de VBA y su modelo de objetos. Este modelo de objetos representa cada una de las partes de Excel en donde la aplicación está fundamentada por el objeto application. En ese sentido, los libros de Excel son representados por el objeto Work Books y las hojas de cálculo de cada libro por el objeto Work Sheets.

De esta manera, cada hoja está compuesta por celdas y en VBA se accede a una celda o un rango en particular de celdas a través del objeto range o self. Es importante recordar que cada uno de los objetos en VBA cuenta con una serie de propiedades y métodos que permiten crear las instrucciones necesarias dentro de una macro como tal.

¿Como grabar una macro?

Entrega de un video

Desarrollo de un reto sobre una acción en particular (opción abierta).